

Lab: Discrete model-based RL in decision-making tasks

March 11, 2025

1 The goal of this practical

Today we are going to get acquainted with discrete tabular model-based reinforcement learning, and more specifically with the Value Iteration algorithm [1]. We will do so in a discretized version of the Tolman detour task proposed by Martinet et al. [2].

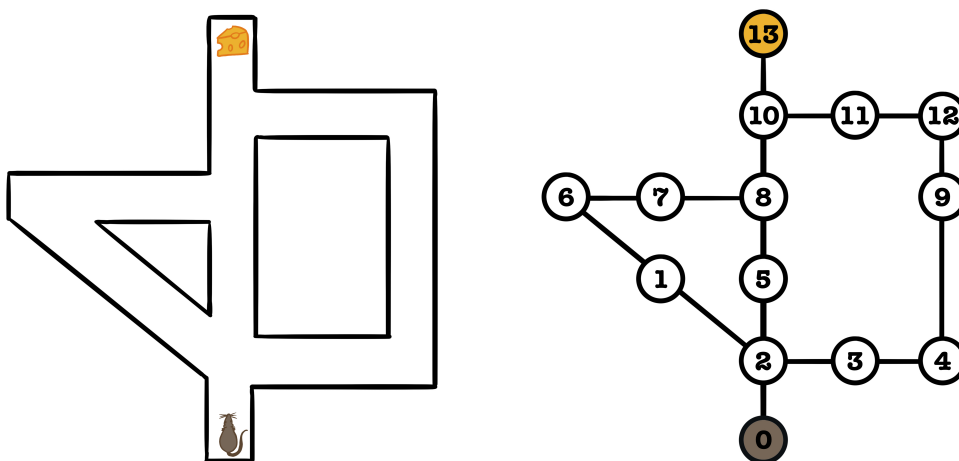


Figure 1: The Tolman detour task (left) in which the rat needs to find the shortest path to reach the food reward. Its discretized version (right) consists of 14 states. The agent is placed in state 0, while the reward is in state 13. In every state the agent has 3 available actions: "LEFT", "UP" and "RIGHT". Actions that would drive the agent out of the maze will simply result in staying in place. Once the agent reached state 13, any action will be rewarded, and the agent will be returned to the initial state.

The code to the maze is in the file *environments.py*. In this environment an episode is defined from the moment the agent enters the maze until the reward acquisition. We will run 200 episodes in each experiment, to make sure that our agents have sufficient time to learn.

2 Quick recap: model-free RL

As we remember from two weeks ago, classical model-free reinforcement learning agents compute a discounted expected reward value (or a so-called Q-value) in order to efficiently estimate the quality of each state-action couple. Given its recursive nature, there exists a closed formula that one can use to compute the Q-values: the Bellmann equation.

What's even better is that one can approximate Q-values on the fly, updating them based on real-time experiences, and without losing the guarantee of convergence (as long as certain criteria hold true of course). This phenomenon is at the core of Q-learning, which updates its Q-values using the formula

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha[r_t(s, a) + \gamma \max_{a' \in \mathcal{A}} Q_t(s', a') - Q_t(s, a)]$$

where γ is the discount factor and α is the learning rate. Once the Q-values are computed, decision making is quite simple: the agent will pick the action with the highest corresponding Q-value with a probability of $(1 - \epsilon)$ and a random action otherwise. This is what we call an epsilon-greedy policy.

Exercise 1 Open *agents.py* and look for the function *Q-learning()* belonging to the class *MFagent*. Code the update rule. Once you're done, run *main.py*. You should see a curve corresponding to the reward rates achieved by the agent. The ideal reward rate in this task should be about 1.67.

3 Model-based reinforcement learning

So what about model-based reinforcement learning? The big difference is that these agents create a virtual model of the environment, which, in our case, will consist of a reward- and a transition function. If you're interested in how these functions are updated feel free to check out the *update_model()* method of the *MBagent*, but essentially what we do here is simply averaging the outcomes of the last 10 transitions for every state and action. Imagine a rolling window, smoothly averaging the reward and transition history – that's exactly what's happening here.

Once we have a reward- and a transition function, however, things can get really fun. Since this virtual model of the environment condenses all of our knowledge of the environment, we can simply use it to fine tune the Q-values, simply following the rule:

$$Q_{t+1}(s, a) = R_t(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s, a, s') \max_{a' \in \mathcal{A}} Q_t(s', a')$$

where $R(s, a)$ is the reward function and $T(s, a, s')$ is the transition function. After every step, we can simply iterate this equation over all states and all actions until our Q-values converge (below a threshold θ). This is called **Value Iteration**.

Exercise 2 In *agents.py* now look for the function *value_iteration()* belonging to the class *MBagent*. Code the update rule. This time you'll have to write multiple lines! Once you're done, you can uncomment the section "The MB agent" in *main.py*, and run it one more time. Now you'll see both the MF and the MB agents' reward rates side by side.

4 But why would we ever use MB-RL

Look at the output on your console. As you might have noticed, we measured the amount of time each agent took to solve this task. And it is not looking great for the model-based agent. Can you guess why it takes so long?

But if this agent is so very slow, then what is even the point of using MB-RL? Why don't we just stick with our handy Q-learning?

Exercise 3 In *main.py*, right where we define the experimental parameters, change the *nr_of_runs* to 20, and re-run the experiment. Also, I highly recommend commenting out the 2 instances of *print()* down where we measured the elapsed time before launching the code. Let's take a look at the reward rates produced this way.

Would you look at that! The MB agent basically demolishes the MF one, and especially in the early game! Why is that?

5 Let's take this a step further

So the MB-RL agent seems to learn quickly and efficiently when we put it into a novel environment. But what happens when it's already used to its surroundings, but then a drastic change takes place?

Won't it just get stuck? This is the question that Martinet and colleagues answered using the detour task.

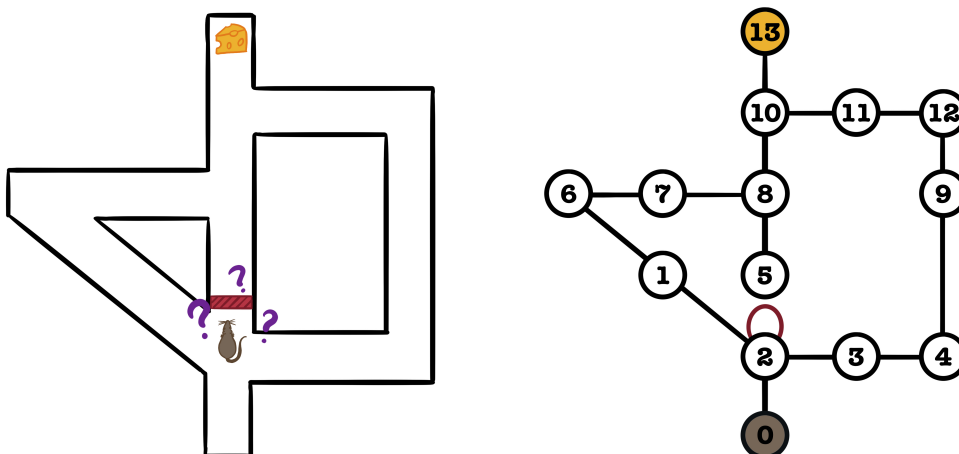


Figure 2: Blocking the middle corridor in the Tolman detour task (left). To simulate this, we can simply forbid the transition between states 2 and 5 in our discretized model. Now if the agent decides to go "UP" in state 2, it will simply find itself in the same state once again.

Before we get down to coding, make a guess. Which agent can adapt better to a change in the environment? Why?

Excercise 4 In *main.py*, right where we define the experimental parameters, change the *obstruct_corridor* to, let's say, 100. This means that after 100 episodes we will suddenly block the central corridor. Before launching *main()* again, I recommend uncommenting the line at the bottom of *main()*, right above *plt.show()*.

Naturally after the central path is blocked, reward rates will drop. Now even the best possible policy could only achieve a reward rate of 1.25 (marked with a second dashed line on your plot). But let's take a look at how the agents fared. Which agent adapted better? Is this what you expected? Can you explain why it happened the way it did?

6 Time to combine things

So MF-RL is significantly quicker and computationally lighter, but MB-RL seems to be much more precise and adaptive. So what about us, humans? Do we seem to use more MF or MB-RL? This is a fascinating question, and the answer is: probably both! Both algorithms have their own strengths and weaknesses, their own moments where they can shine. Can you name some examples of both?

Of course, combining MF and MB solutions is not as straightforward as it sounds. We can come up with infinitely many solutions to balance the two. Here I created a new class, called *CombinedAgent* that uses both of our previous algorithms in parallel to optimize its computations. I used some very simplistic rules to make it work: during action selection, both agents propose a decision, and the one with the higher corresponding Q-value will win; and as for learning, I allow the MF algorithm to learn in every step, the MB one, however, can only update its Q-values if the MF-update was significantly large.

Excercise 5 You can now uncomment the "*Combined agent*" section at the bottom of the *main()*. If you want to launch it as-is, I recommend that you comment out the *print()* function that displays the time it requires. Alternatively, if you're interested in its speed compared to the others, then make sure to print the times for all 3 agents, and set *nr_of_runs* to 1 at the top of *main()*. Play around and see how it fares compared to the others.

Does this combined agent manage to beat the others? Or is it left behind? Of course this agent is nothing but a quick demonstration of what could be possible with a combined algorithm, and it is by no means optimal. So that leads me to...

Exercise 6 for you: try and come up with a smarter way to integrate the 2 agents. You can measure time and computational load, you can compare information content, you can do whatever you wish! Surprise me. Open *combined_agent.py* and change it so that you create a brand new combined MF-MB agent. Have fun!

References

- [1] R. S. Sutton and A. Barto, *Reinforcement learning : an introduction*. Cambridge, Ma ; London: The Mit Press, 2018.
- [2] L.-E. Martinet, D. Sheynikhovich, K. Benchenane, and A. Arleo, “Spatial learning and action planning in a prefrontal cortical network model,” *PLoS Computational Biology*, vol. 7, p. e1002045, May 2011.